

Detecção de Objetos

PROF. CESAR HENRIQUE COMIN

Sumário

- [Conceitos gerais](#)
- [Métodos básicos de detecção](#)
- [Deteção usando redes neurais](#)
- [Principais parâmetros de detecção: limiar de confiança e de IoU](#)
- [O modelo YOLO](#)
- [Deteção de pose](#)

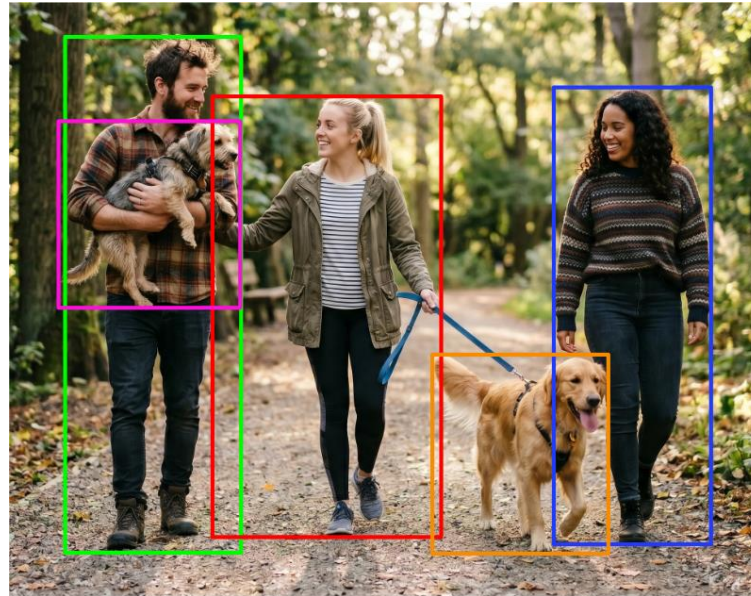
Detecção de objetos

A detecção de objetos consiste em duas tarefas realizadas de forma simultânea:

1. Extração das coordenadas de caixas delimitadoras (bounding box) dos objetos relevantes das imagens
2. Classificação do conteúdo de cada caixa delimitadora

Detecção de objetos

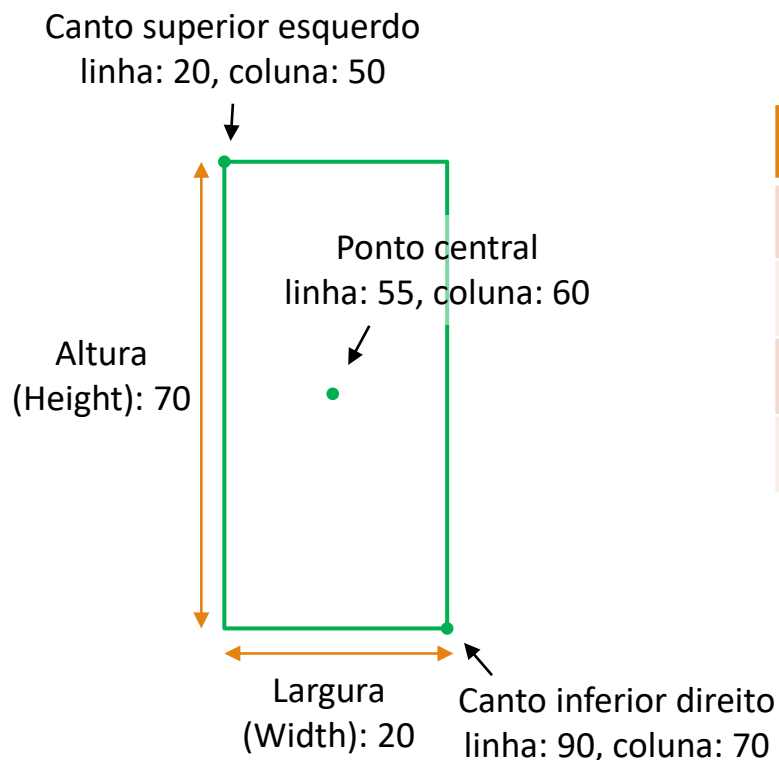
Por exemplo:



Coordenadas	Classe
[12, 34, 820, 140]	Pessoa
[70, 180, 790, 535]	Pessoa
[68, 670, 801, 920]	Pessoa
[93, 29, 430, 139]	Cachorro
[515, 528, 820, 702]	Cachorro

Detecção de objetos

As coordenadas podem estar em diferentes formatos:

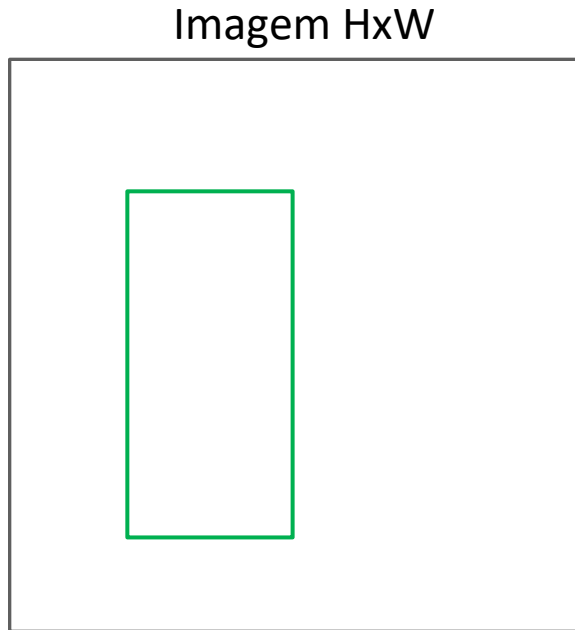


Formato	Valores
XYXY	[50, 20, 70, 90]
YXYX	[20, 50, 90, 70]
XYWH	[50, 20, 20, 70]
CXCYWH	[60, 55, 20, 70]

Note que
X <-> Coluna
Y <-> Linha

Detecção de objetos

- Também é comum as coordenadas serem normalizadas pelo tamanho da imagem:



$$[x1, y2, x2, y2] \rightarrow [x1/W, y2/H, x2/W, y2/H]$$

- Isso torna as coordenadas independentes da resolução da imagem

Métodos básicos de detecção

- Para entender o funcionamento de métodos atuais, iremos primeiro considerar o funcionamento de dois métodos clássicos:
 1. Detecção por janela deslizante
 2. Detecção a partir de regiões propostas

Detecção Utilizando Janela Deslizante

Detecção Utilizando Janela Deslizante

Queremos encontrar todas as pessoas nessa cena



Detecção Utilizando Janela Deslizante

A estratégia é deslizar uma janela sobre a imagem, e tentar classificar cada imagem sob a janela, verificando se há uma pessoa na janela



Detecção Utilizando Janela Deslizante

A estratégia é deslizar uma janela sobre a imagem, e tentar classificar cada imagem sob a janela, verificando se há uma pessoa na janela



Detecção Utilizando Janela Deslizante

A estratégia é deslizar uma janela sobre a imagem, e tentar classificar cada imagem sob a janela, verificando se há uma pessoa na janela



Detecção Utilizando Janela Deslizante

A estratégia é deslizar uma janela sobre a imagem, e tentar classificar cada imagem sob a janela, verificando se há uma pessoa na janela



Detecção Utilizando Janela Deslizante

As pessoas possuem diferentes tamanhos, como podemos tratar essa variação de tamanho?



Detecção Utilizando Janela Deslizante

As pessoas possuem diferentes tamanhos, como podemos tratar essa variação de tamanho?

Diminuímos a imagem e repetimos a busca. O tamanho da janela é mantido fixo.



Detecção Utilizando Janela Deslizante

Aplicamos a janela deslizante em cada nível da pirâmide multiresolução da imagem



Detecção Utilizando Janela Deslizante

Para classificarmos as imagens encontradas em cada janela, precisamos primeiro treinar um classificador

Para isso, criamos uma base de imagens de treinamento

Imagens de treinamento contendo pessoas (exemplos positivos)



Imagens de treinamento sem pessoas (exemplos negativos)



Detecção Utilizando Janela Deslizante

Dada uma nova imagem, na qual queremos detectar pessoas:

- Para cada região sob a janela deslizante, classificamos a região utilizando o classificador treinado na base de imagens
- No final do procedimento, temos associado a cada região analisada a informação “pessoa” ou “não pessoa”

Detecção Utilizando Janela Deslizante

- O resultado, em geral, inclui uma grande quantidade de janelas, pois diversas janelas próximas acabam sendo associadas a cada pessoa



Detecção Utilizando Janela Deslizante

- Para remover janelas sobrepostas, utilizamos o método de supressão de não-máximos (non-maximum suppression)
 1. Obtenha a janela J com maior probabilidade de conter uma pessoa
 2. Elimine as janelas possuindo intersecção com J maior do que um limiar IoU_t
 3. Armazene J numa lista final de janelas e remova-a da imagem
 4. Repita 1 a 3 até que não haja mais janelas a serem analisadas



Detecção Utilizando Janela Deslizante

- Para remover janelas sobrepostas, utilizamos o método de supressão de não-máximos (non-maximum suppression)
 1. Obtenha a janela J com maior probabilidade de conter uma pessoa
 2. Elimine as janelas possuindo intersecção com J maior do que um limiar IoU_t
 3. Armazene J numa lista final de janelas e remova-a da imagem
 4. Repita 1 a 3 até que não haja mais janelas a serem analisadas



Detecção Utilizando Janela Deslizante

- Para remover janelas sobrepostas, utilizamos o método de supressão de não-máximos (non-maximum suppression)
 1. Obtenha a janela J com maior probabilidade de conter uma pessoa
 2. Elimine as janelas possuindo intersecção com J maior do que um limiar IoU_t
 3. Armazene J numa lista final de janelas e remova-a da imagem
 4. Repita 1 a 3 até que não haja mais janelas a serem analisadas



Detecção Utilizando Janela Deslizante

- Para remover janelas sobrepostas, utilizamos o método de supressão de não-máximos (non-maximum suppression)
 1. Obtenha a janela J com maior probabilidade de conter uma pessoa
 2. Elimine as janelas possuindo intersecção com J maior do que um limiar IoU_t
 3. Armazene J numa lista final de janelas e remova-a da imagem
 4. Repita 1 a 3 até que não haja mais janelas a serem analisadas



Detecção Utilizando Janela Deslizante

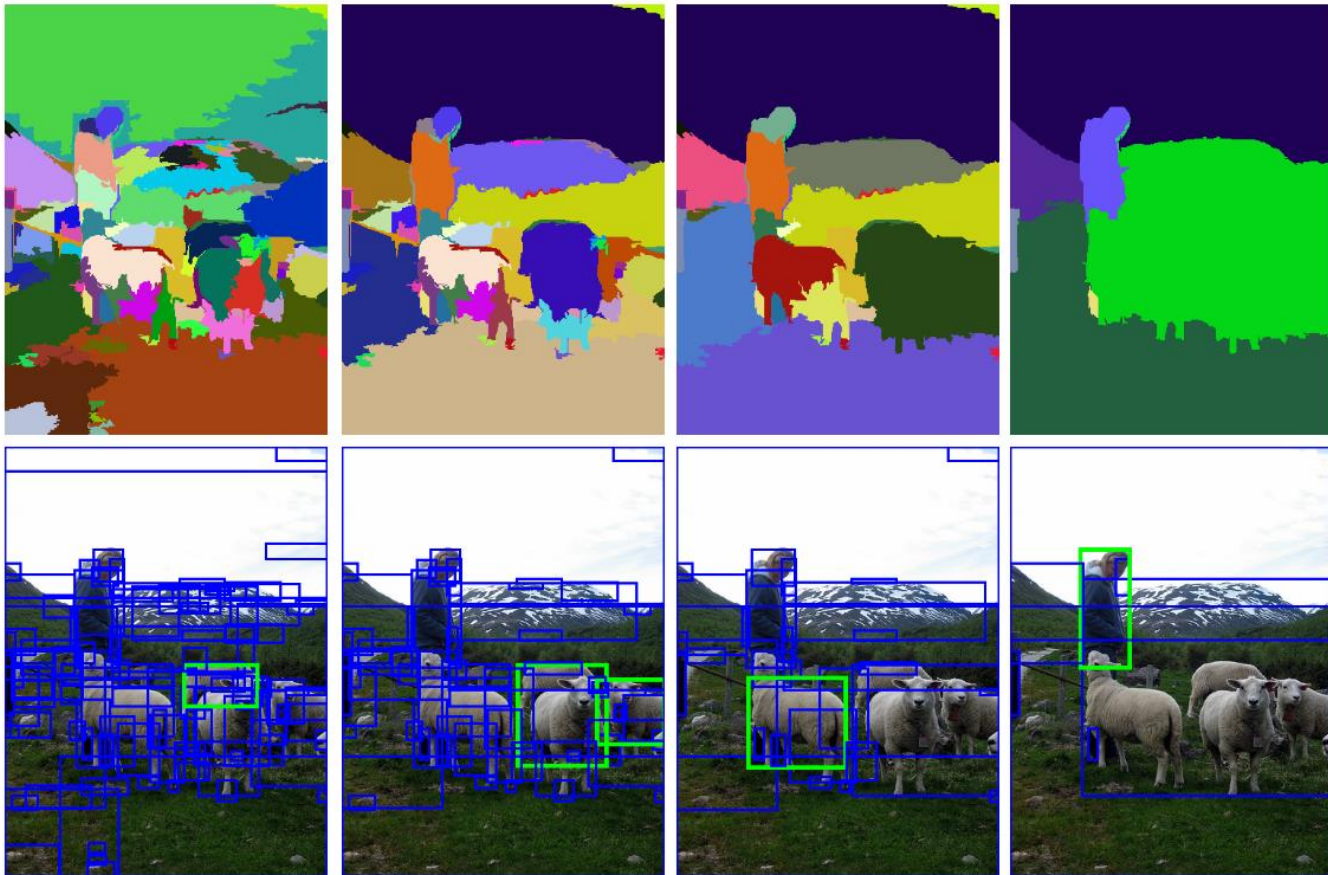
- Para remover janelas sobrepostas, utilizamos o método de supressão de não-máximos (non-maximum suppression)
 1. Obtenha a janela J com maior probabilidade de conter uma pessoa
 2. Elimine as janelas possuindo intersecção com J maior do que um limiar IoU_t
 3. Armazene J numa lista final de janelas e remova-a da imagem
 4. Repita 1 a 3 até que não haja mais janelas a serem analisadas

Problema! O
que fazer?



Detecção a partir de regiões propostas

- Existem métodos que permitem agrupar pixels similares em imagens em diferentes escalas
- Esses métodos podem ser utilizados para "propor" possíveis caixas delimitadoras
- Um classificador é treinado a partir das caixas propostas



Detecção de objetos usando redes neurais

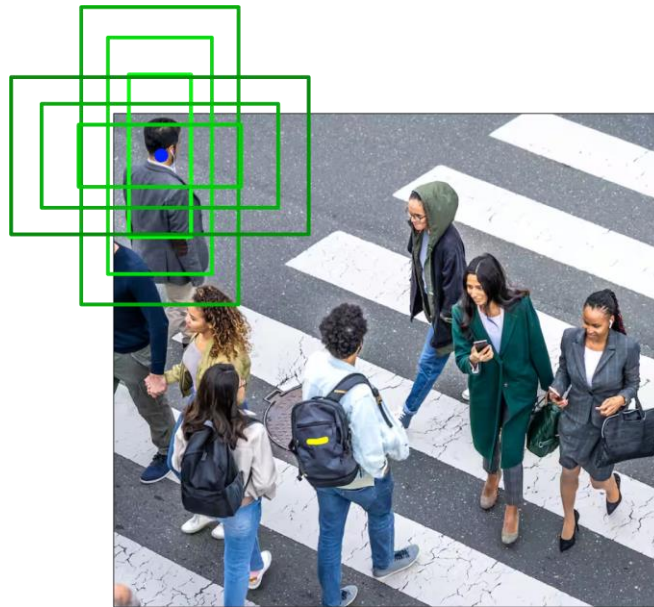
Detecção de objetos usando redes neurais

Uma grade pontos de referência é definida na imagem



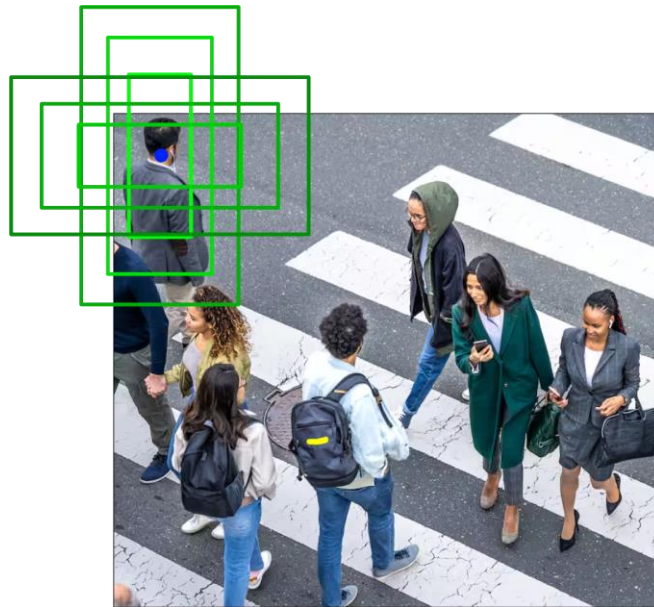
Detecção de objetos usando redes neurais

- Para cada ponto, um respectivo conjunto de janelas é definido
- As janelas possuem:
 1. Diferentes escalas (tamanhos)
 2. Diferentes razões de aspecto
- Essas caixas são chamadas de **caixas âncora**



Detecção de objetos usando redes neurais

- Tipicamente, são definidas caixas em 3 escalas e 3 razões de aspecto
- Total de caixas: $9 \cdot N$, onde N é o número de posições espaciais (número de pontos)



Detecção de objetos usando redes neurais

- Para treinar um método de detecção, o primeiro passo é anotar o conjunto de dados

img1.jpg



Em um arquivo de texto chamado img1.txt:

```
0, 0.12, 0.34, 0.82, 0.14  
0, 0.70, 0.18, 0.79, 0.53  
0, 0.68, 0.67, 0.80, 0.92  
4, 0.93, 0.29, 0.43, 0.13  
4, 0.51, 0.52, 0.82, 0.70
```

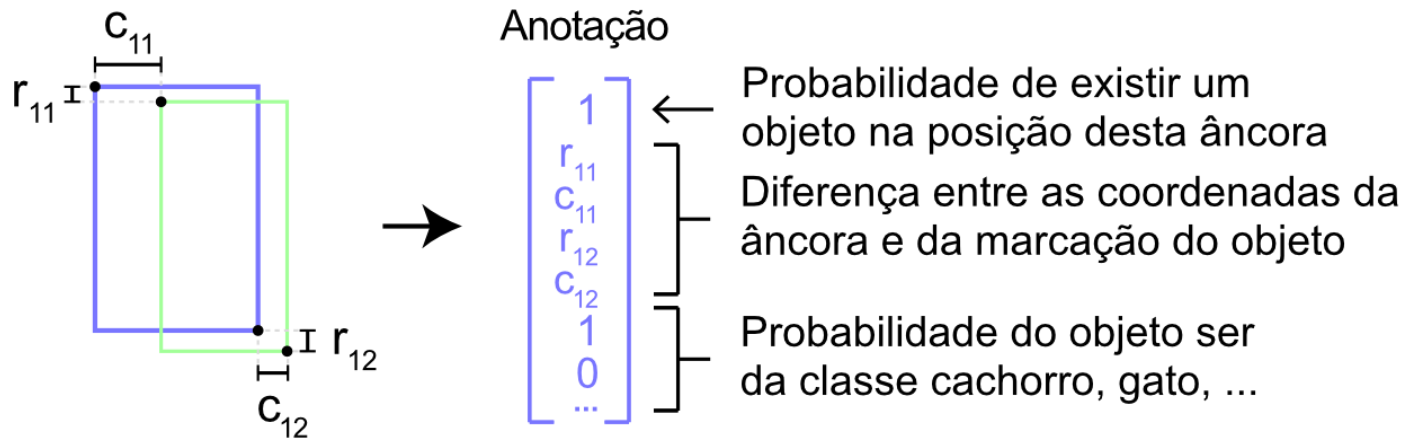
onde a primeira coluna representa a classe (0: pessoa, 4: cachorro) e os demais valores a posição normalizada da caixa

Detecção de objetos usando redes neurais

- Ferramentas de código aberto populares para anotação de imagens:
 - CVAT
 - Label Studio
 - LabelMe

Detecção de objetos usando redes neurais

- Para cada objeto anotado manualmente, é encontrada a caixa âncora com maior intersecção, e os seguintes valores são calculados:

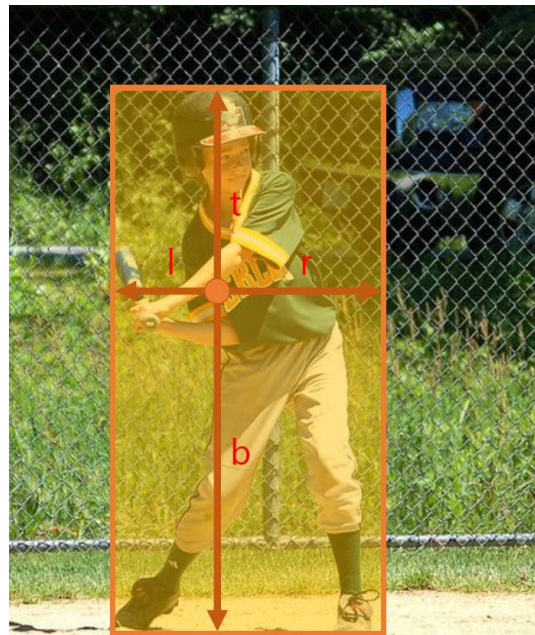


Detecção de objetos usando redes neurais

- Após a anotação do conjunto de dados, a rede é treinada para gerar $k * N * (4 + 1 + C)$ valores para cada imagem
 - k : Número de caixas âncora para cada posição
 - N : Número de posições das caixas
 - C : Número de classes
 - 4: Valores de ajuste das coordenadas de cada caixa
 - 1: presença/ausência de objeto

Detecção de objetos usando redes neurais

- Muitos detectores atuais abandonaram a ideia de caixa âncora
- Para cada ponto de referência (os mesmos usados nas caixas), um vetor de 4 valores (l, t, r, b) é retornado. Ele possui as distâncias do ponto até as bordas esquerda, superior, direita e inferior da caixa delimitadora real do objeto
- Portanto, o modelo retorna $N * (4 + 1 + C)$ valores para cada imagem

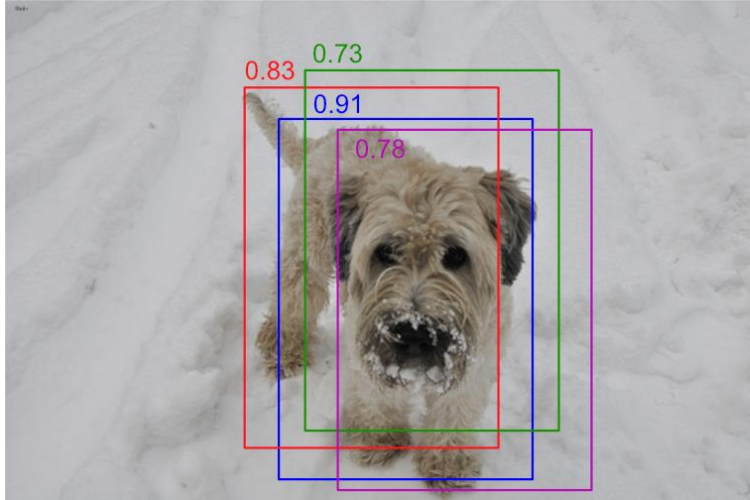


Aplicação de um modelo de detecção

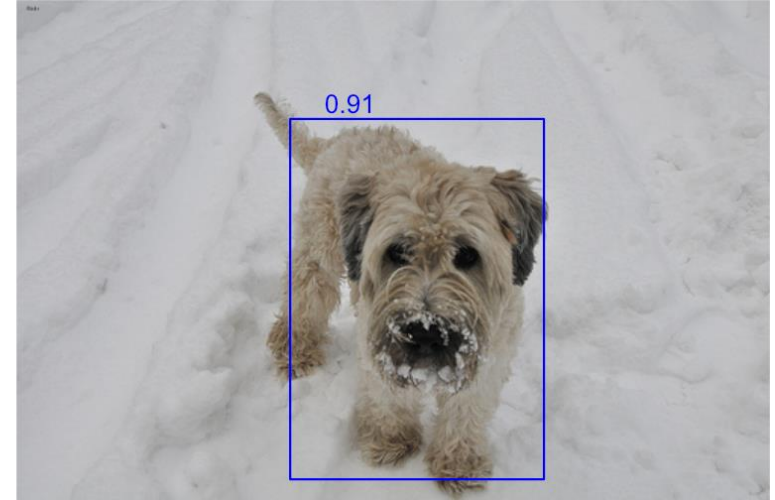
- Na aplicação do modelo, os parâmetros principais são:
 - Limiar de confiança: Probabilidade a partir da qual uma detecção é considerada como válida
 - Limiar de IoU: Detecções com sobreposição maior do que esse valor são removidas

Aplicação de um modelo de detecção

- Na aplicação do modelo, os parâmetros principais são:
 - Limiar de confiança: Probabilidade a partir da qual uma detecção é considerada como válida
 - Limiar de IoU: Deteções com sobreposição maior do que esse valor são removidas

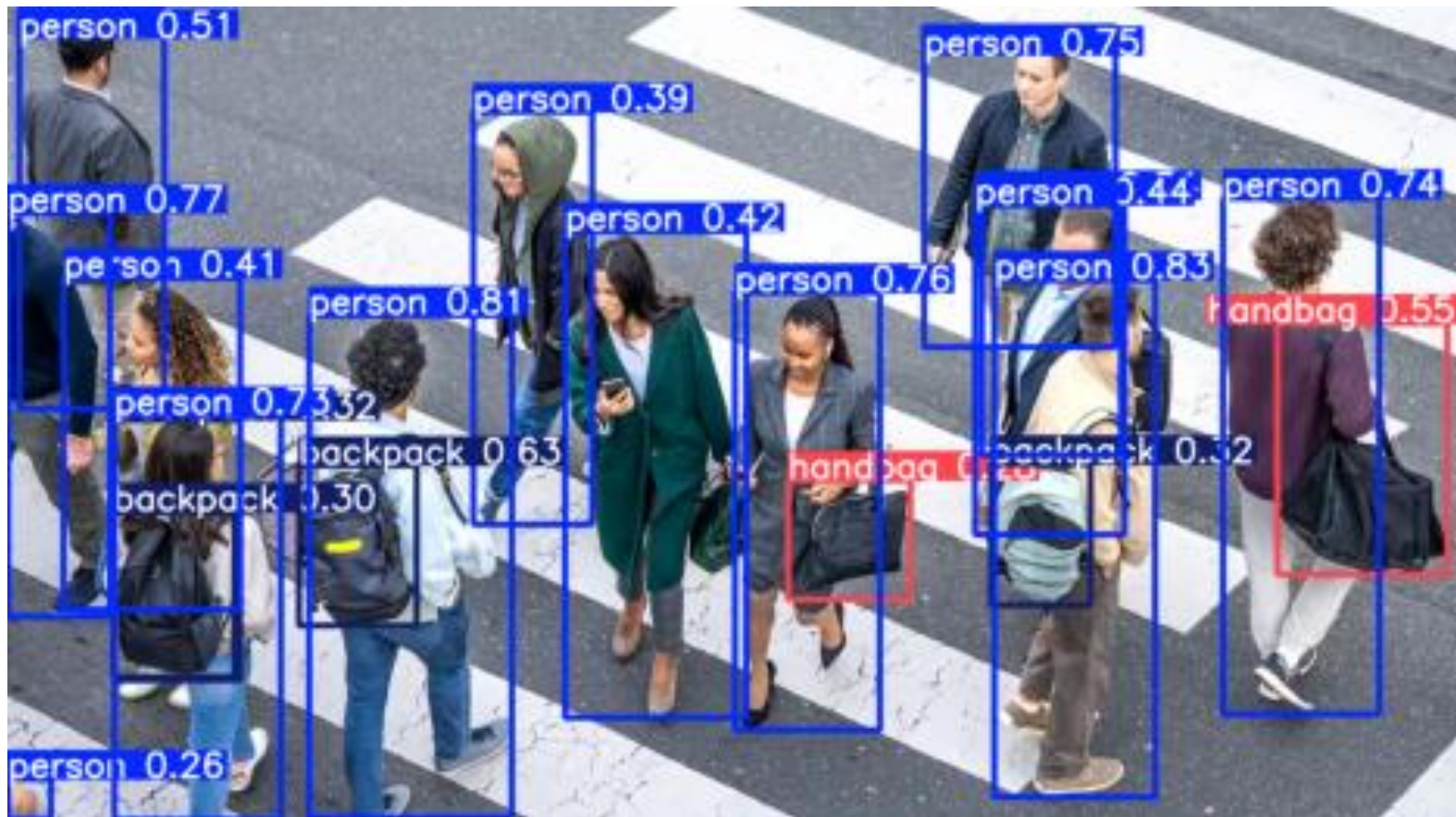


Supressão de
não máximos



Aplicação de um modelo de detecção

Exemplo de resultado. Os valores ao lado do nome da classe indicam a probabilidade



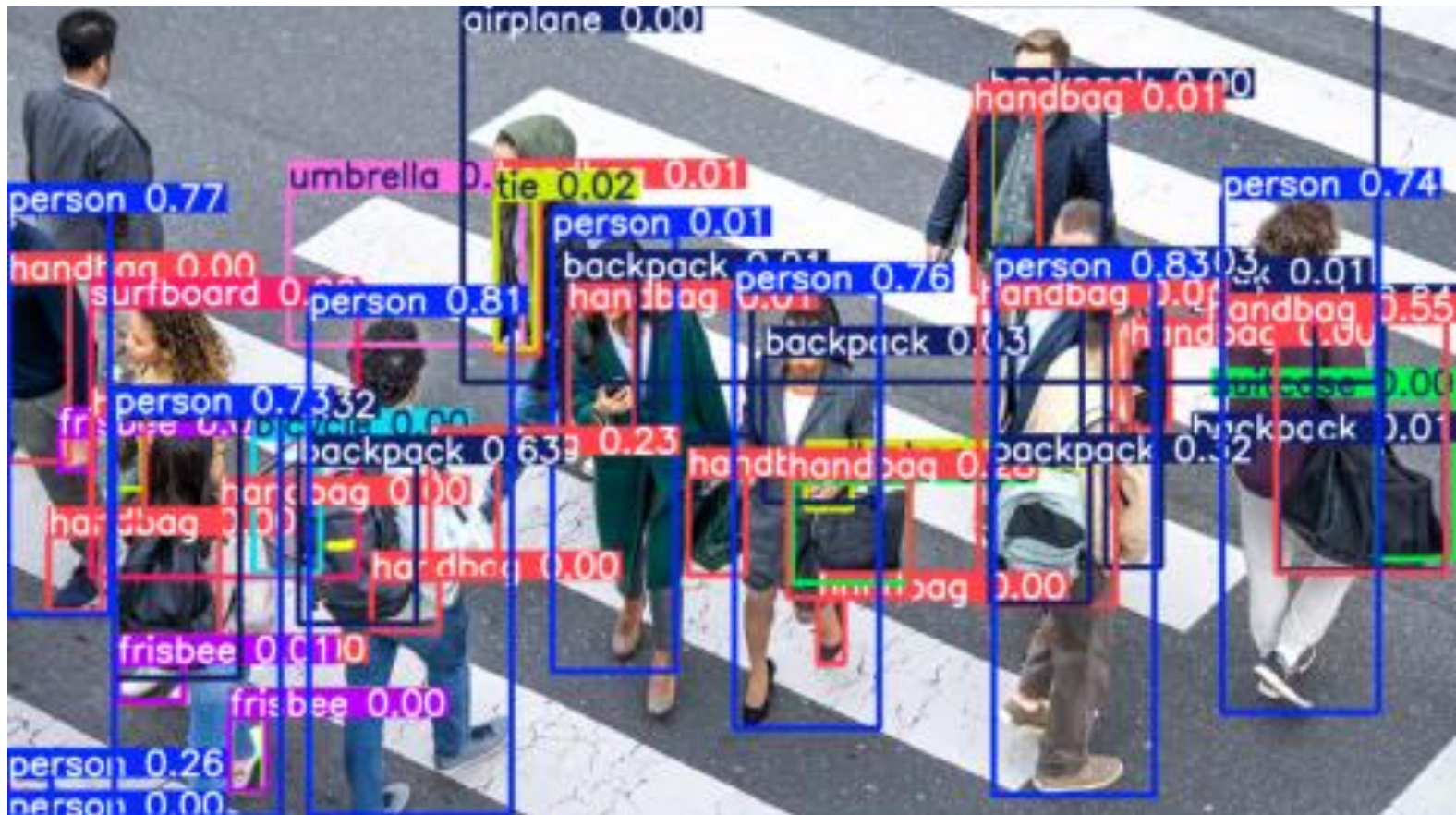
Aplicação de um modelo de deteção

Resultado utilizando um limiar de confiança/probabilidade de 0.00001



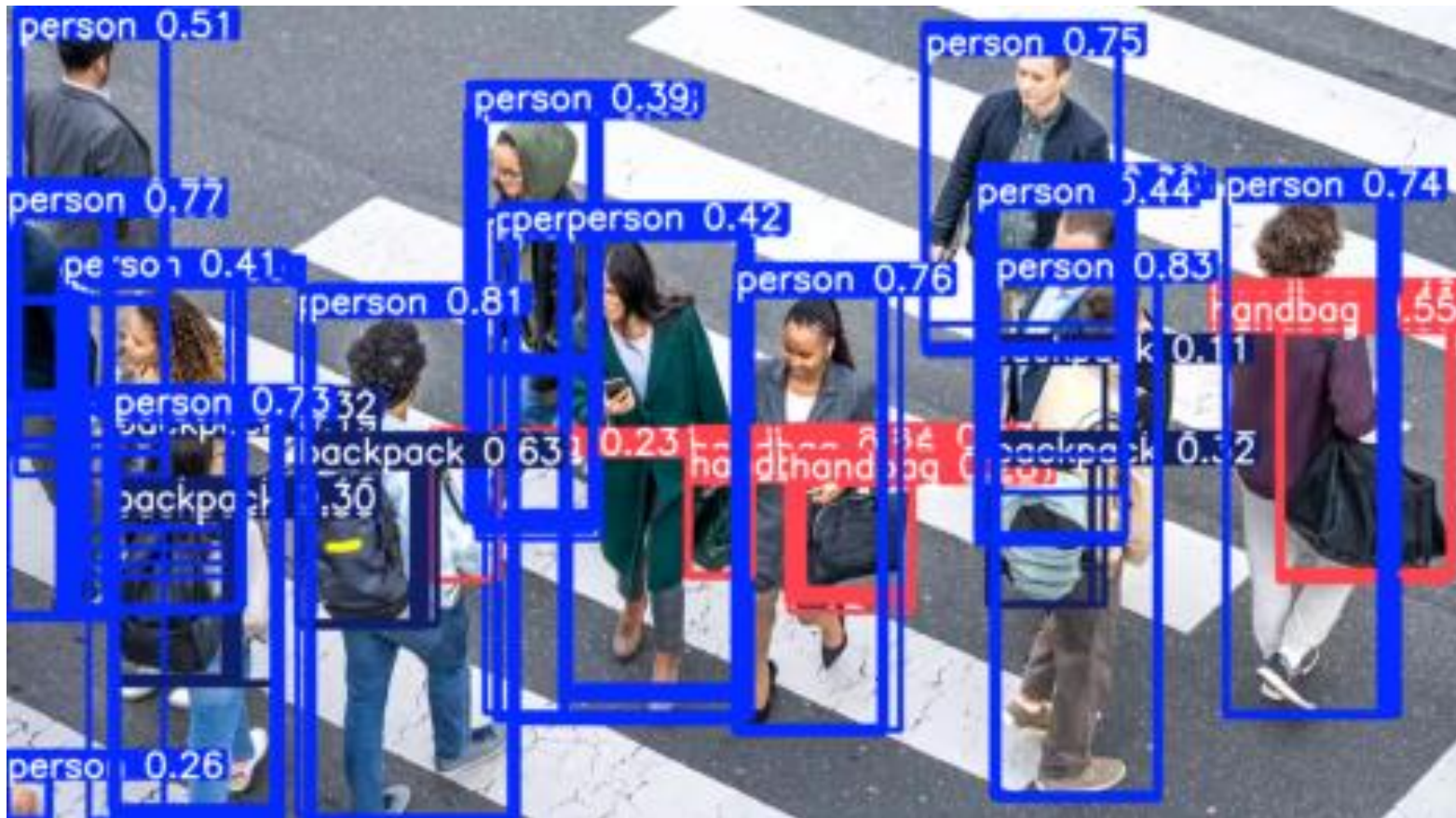
Aplicação de um modelo de deteção

Resultado utilizando um limiar de confiança/probabilidade de 0.001



Aplicação de um modelo de detecção

Limiar de confiança 0.1 e limiar de IoU 1.0. Caixas com sobreposição são detectadas



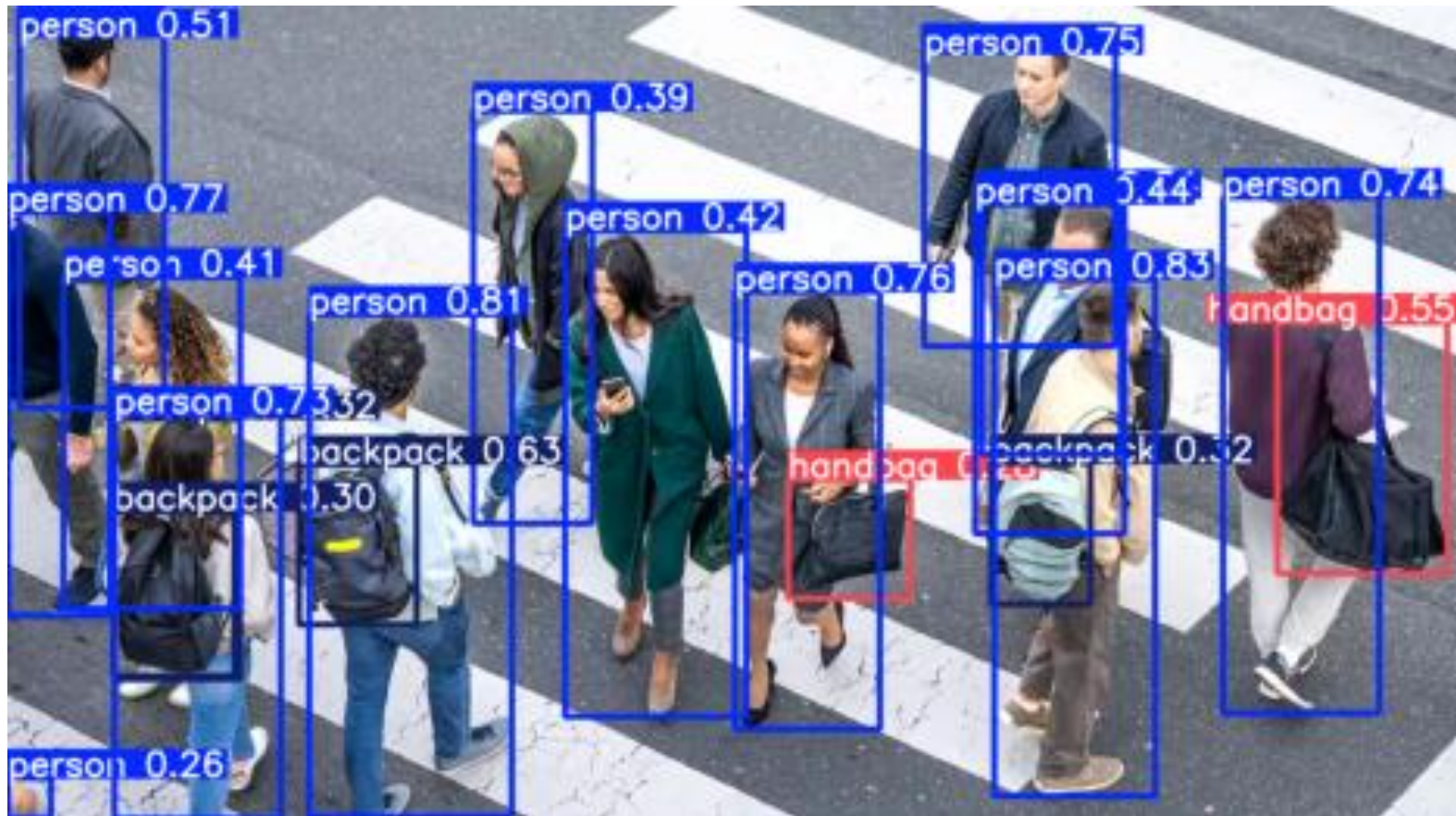
Aplicação de um modelo de detecção

Limiar de confiança 0.1 e limiar de IoU 0.0. Pessoas próximas não são detectadas



Aplicação de um modelo de detecção

Limiar de confiança 0.25 e limiar de IoU 0.7



O modelo YOLO

Atualmente, os modelos mais populares de detecção de objetos são da família YOLO

Modelo	Author	Licença
YOLOv1	Joseph Redmon	MIT
YOLOv2 / YOLO9000	Joseph Redmon, Ali Farhadi	MIT
YOLOv3	Joseph Redmon, Ali Farhadi	MIT
YOLOv4	Alexey Bochkovskiy et al., Academia Sinica, Taiwan	GPL-3.0
YOLOv5	Ultralytics	GPL-3.0
YOLOv6	Meituan Inc.	GPL-3.0
YOLOv7	Chien-Yao Wang et al., Academia Sinica, Taiwan	GPL-3.0
YOLOv7	MultimediaTechLab, reimplementação	MIT
YOLOv8	Ultralytics	AGPL-3.0
YOLOv9	Chien-Yao Wang et al., Academia Sinica, Taiwan	MIT
YOLOv10	Ultralytics	AGPL-3.0
YOLOv11	Ultralytics	AGPL-3.0
YOLO12	Yunjie Tian et al., SUNY	AGPL-3.0
YOLO26	Ultralytics	AGPL-3.0
YOLOX	Megvii	Apache-2.0

O modelo YOLO

- Uma biblioteca Python muito popular para a tarefa é a *ultralytics*, desenvolvida pela empresa de mesmo nome
- Os modelos da empresa são, em geral, treinados no conjunto de dados COCO (Common Objects in Context)

O modelo YOLO

- Uma biblioteca Python muito popular para a tarefa é a *ultralytics*, desenvolvida pela empresa de mesmo nome
- Os modelos da empresa são, em geral, treinados no conjunto de dados COCO (Common Objects in Context)

O modelo YOLO

- Importante! A implementação atual do modelo possui o parâmetro `imgsz`, que define o tamanho da imagem de entrada. O padrão é `imgsz = 640`
- A imagem é redimensionada de forma que o maior lado (altura ou largura) seja igual a `imgsz`, o outro lado é redimensionado de forma a manter a razão de aspecto da imagem

O modelo YOLO

Notebook "**1 - O modelo YOLO**"

Detecção de pose

- A tarefa de detecção de pose consiste em estimar a posição de um conjunto de uma ou mais pessoas em uma imagem
- Isso é feito através da identificação de pontos-chave (keypoints) das pessoas



Detecção de pose

- A tarefa é muito similar à de detecção de objetos
- É definido um conjunto de pontos-chave representando pontos específicos do corpo humano
- Por exemplo:

Nariz	Punho Esquerdo
Olho Esquerdo	Punho Direito
Olho Direito	Quadril Esquerdo
Orelha Esquerda	Quadril Direito
Orelha Direita	Joelho Esquerdo
Ombro Esquerdo	Joelho Direito
Ombro Direito	Tornozelo Esquerdo
Cotovelo Esquerdo	Tornozelo Direito
Cotovelo Direito	

Detecção de pose

Para cada imagem, a rede prevê $k * N * (1 + 4 + 3 * p)$ valores

- k : Número de caixas âncora para cada posição
- N : Número de posições das caixas
- 1: Probabilidade da caixa conter uma pessoa
- 4: Coordenadas da caixa delimitadora
- p : Número de pontos-chave suportados pelo modelo
- 3: Para cada ponto chave são retornados 3 valores: a posição (x, y) do ponto e a probabilidade do ponto estar visível

Detecção de pose

Duas bibliotecas são populares para a tarefa de detecção de pose:

- ultralytics: Detecta os 17 pontos-chave mostrados no slide anterior
- MediaPipe: Detecta 32 pontos chave

Detecção de pose

Duas bibliotecas são populares para a tarefa de detecção de pose:

- ultralytics: Detecta os 17 pontos-chave mostrados no slide anterior
- mediapipe: Detecta 32 pontos chave

